Laboratório Prolog · Aula 13

andre mulati · RA 24011149-2 · Turma B
Gerado em 20/10/2026, 20:11:34 · tempo registrado no cronômetro: 25:45

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

?- linguagem(A).
linguagem(prolog)
linguagem(scheme)
linguagem(java)
?- paradigma(prolog,A).
paradigma(prolog,logico)
?- paradigma(A,funcional).
paradigma(scheme,funcional)
?- paradigma(java,logico).
false.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(java,A).
   paradigma(java,orientado_a_objetos)
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

A diferença é que o segundo exemplo tem uma consulta extra:

Primeiro exemplo: termina em false. porque java não está associado ao paradigma logico.

Segundo exemplo: depois do false., faz uma nova consulta: paradigma(java,A)., que retorna orientado_a_objetos.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não encontrou esse fato na sua base de conhecimento.
false indica que não existe um fato ou regra que prove paradigma(java, logico) no programa. Isso não significa que Java não seja uma linguagem lógica em sentido geral, mas que, na base apresentada, Java está associado a orientado_a_objetos, e não a logico.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
  mae(joana,jake)
?- mae(jake,joana).
  false.
?- pai(maria,jake).
  false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
  mae(joana,jake)
  mae(joana,ana)
?- pai(A,jake).
  pai(jorge,jake)
?- mulher(A).
  mulher(lucia)
  mulher(marta)
  mulher(tereza)
  mulher(joana)
  mulher(bia)
  mulher(ana)
  mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
  mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
  pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
  mae(lucia,tereza),mulher(tereza)
  mae(marta,joana),mulher(joana)
  mae(tereza,bia),mulher(bia)
  mae(joana,ana),mulher(ana)

Saída do familia.pl

```
--- 1. sem variavel: a resposta e true ou false
?- mae(joana,jake).
   mae(joana,jake)
?- mae(jake,joana).
   false.
?- pai(maria,jake).
   false.
--- 2. com variavel: o Prolog procura os valores que tornam a consulta verdadeira
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
```

```
?- mulher(A).
   mulher(lucia)
   mulher(marta)
   mulher(tereza)
   mulher(joana)
   mulher(bia)
   mulher(ana)
   mulher(dora)
--- 3. conjuncao (virgula = E): as duas metas valem com o mesmo valor
?- mae(joana,A),homem(A).
   mae(joana,jake),homem(jake)
?- pai(vern,A),mulher(A).
   pai(vern,joana),mulher(joana)
?- mae(A,B),mulher(B).
   mae(lucia,tereza),mulher(tereza)
   mae(marta,joana),mulher(joana)
   mae(tereza,bia),mulher(bia)
   mae(joana,ana),mulher(ana)
?- mae(joana,A).
   mae(joana,jake)
   mae(joana,ana)
?- pai(A,jake).
   pai(jorge,jake)
?- mae(A,B),homem(B).
   mae(lucia,vern),homem(vern)
   mae(marta,paulo),homem(paulo)
   mae(joana,jake),homem(jake)
   mae(dora,leo),homem(leo)
?- homem(A),mulher(A).
   false.
?- mae(joana,jake),mae(joana,ana).
   mae(joana,jake),mae(joana,ana)
Ready
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Significa: procurar todas as mães que têm filhas do sexo feminino, ou seja, encontrar pares em que Mae é mãe de Filha e Filha é mulher.

Teve 4 respostas

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O Prolog usa a hipótese do mundo fechado: considera falso tudo aquilo que não pode provar a partir dos fatos e regras existentes na base de conhecimento.

Isso pode ser um problema em situações reais porque uma informação ausente não é necessariamente falsa.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

lucia

marta

tereza

joana

bia

ana

dora

A ordem em que os fatos mulher(...) foram escritos na base de conhecimento. O Prolog procura os fatos de cima para baixo e retorna as soluções nessa sequência.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

```
--- avo: duas regras progenitor encadeadas
?- avo(A,jake).
  avo(marta,jake)
  avo(vern,jake)
?- avo(osvaldo,A).
  avo(osvaldo,joana)
  avo(osvaldo,paulo)
  avo(osvaldo,bia)
--- irmao, tio e primo
?- irmao(jake,A).
  irmao(jake,ana)
?- tio(A,jake).
  tio(paulo,jake)
?- primo(leo,A).
  primo(leo,jake)
  primo(leo,ana)
--- ancestral (recursiva)
?- ancestral(osvaldo,A).
  ancestral(osvaldo,vern)
  ancestral(osvaldo,tereza)
  ancestral(osvaldo,joana)
  ancestral(osvaldo,paulo)
  ancestral(osvaldo,jake)
  ancestral(osvaldo,ana)
  ancestral(osvaldo,leo)
  ancestral(osvaldo,bia)
?- ancestral(A,jake).
  ancestral(joana,jake)
  ancestral(jorge,jake)
  ancestral(lucia,jake)
  ancestral(marta,jake)
```

```
  ancestral(osvaldo,jake)
  ancestral(vern,jake)
?- ancestral(jake,A).
  false.
Ready
```

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

falham a 1.ª, a 3.ª e a 4.ª unificações.

Saídas de regras.pl e unificacao.pl

```
--- constantes e variaveis
a  =  a    unifica
a  =  b    NAO unifica
A  =  a    unifica   A = a
--- estruturas
f(A,b)  =  f(a,B)    unifica   A = a   B = b
f(A,A)  =  f(a,b)    NAO unifica
f(A,A)  =  f(a,B)    unifica   A = a   B = a
pai(A,jake)  =  pai(jorge,B)    unifica   A = jorge   B = jake
--- listas: [Cabeca|Cauda]
[A|B]  =  [1,2,3]    unifica   A = 1   B = [2,3]
[A|B]  =  []    NAO unifica
[A,B|C]  =  [a,b,c,d]    unifica   A = a   B = b   C = [c,d]
--- aritmetica NAO e unificacao
2+3  =  5    NAO unifica
f(A,B)  =  f(1,2)    unifica   A = 1   B = 2
g(A,b)  =  g(a,A)    NAO unifica
[A,B|C]  =  [1,2,3,4,5]    unifica   A = 1   B = 2   C = [3,4,5]
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Porque a regra é recursiva e precisa de um caso para começar e outro para continuar.

O caso base evita que a recursão continue indefinidamente sem uma condição de parada.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem essa condição, uma pessoa pode ser considerada irmã de si mesma, desde que tenha os mesmos pais.
A meta X \== Y garante que X e Y sejam diferentes. Ela é necessária para impedir que alguém apareça como seu próprio irmão.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

f(X, X) = f(a, b) — Falha, porque X teria de ser a e b ao mesmo tempo, mas a e b são diferentes.

f(X, X) = f(a, Y) — Sucesso, porque podemos atribuir a a ambas as variáveis.

Valores finais: X = a e Y = a

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, sim. A regra avo(X, Z) permite consultar tanto os avós de alguém quanto os netos de alguém, porque as variáveis podem assumir valores diferentes conforme a consulta.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

A consulta concatena(X, Y, [1, 2, 3]) vai dar 4 respostas, pois existem quatro maneiras de dividir a lista

Saída do listas.pl

```
--- membro: serve para testar E para gerar
?- membro(b,[a,b,c]).
   membro(b,[a,b,c])
?- membro(z,[a,b,c]).
   false.
?- membro(A,[a,b,c]).
   membro(a,[a,b,c])
   membro(b,[a,b,c])
   membro(c,[a,b,c])
--- tamanho
?- tamanho([a,b,c],A).
   tamanho([a,b,c],3)
?- tamanho([],A).
   tamanho([],0)
--- concatena: o mesmo codigo, em varias direcoes
?- concatena([1,2],[3,4],A).
   concatena([1,2],[3,4],[1,2,3,4])
?- concatena(A,[3],[1,2,3]).
   concatena([1,2],[3],[1,2,3])
?- concatena([1],A,[1,2,3]).
   concatena([1],[2,3],[1,2,3])
?- concatena(A,B,[1,2,3]).
   concatena([],[1,2,3],[1,2,3])
   concatena([1],[2,3],[1,2,3])
   concatena([1,2],[3],[1,2,3])
   concatena([1,2,3],[],[1,2,3])
--- inverte
?- inverte([1,2,3],A).
   inverte([1,2,3],[3,2,1])
?- concatena(A,[b,c],[a,b,c]).
   concatena([a],[b,c],[a,b,c])
?- inverte([a,b,c,d],A).
   inverte([a,b,c,d],[d,c,b,a])
?- tamanho([x,y],A).
   tamanho([x,y],2)
?- membro(A,[]).
   false.
Ready
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0). — uma lista vazia tem tamanho zero.

Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. — calcula o tamanho da cauda e soma 1.
Para tamanho([a,b,c], N), fica pendente a cada chamada:

tamanho([a,b,c], N) → calcular o tamanho de [b,c] e somar 1.

tamanho([b,c], M) → calcular o tamanho de [c] e somar 1.

tamanho([c], ...) → calcular o tamanho de [] e somar 1.

tamanho([], 0) → caso base; a recursão termina e os resultados voltam: 1, 2 e 3.

Resultado final: N = 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Porque existem quatro maneiras de dividir [1,2,3] em duas listas, incluindo as listas vazias:

X = [], Y = [1,2,3]

X = [1], Y = [2,3]

X = [1,2], Y = [3]

X = [1,2,3], Y = []

No Scheme, a função tradicional append recebe as listas e retorna a concatenação. Ela não procura automaticamente todas as divisões possíveis quando você fornece o resultado e deixa as entradas desconhecidas. O Prolog consegue fazer isso por meio de unificação, recursão e busca de soluções.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O mecanismo é a recursão com busca e retrocesso (backtracking).

A primeira regra verifica se X é a cabeça da lista.

A segunda regra procura X na cauda, chamando membro novamente.

Quando o usuário pede outra solução (por exemplo, com ; no terminal), o Prolog volta e tenta alternativas até encontrar todas as respostas.

Assim, o Prolog gera a, b e c automaticamente, sem precisar de um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3: unifica X com a expressão simbólica 2 + 3. Não calcula; X = 2+3.

X is 2 + 3: calcula a expressão e retorna X = 5.

fatorial(N, 120): dá erro de instanciação, porque a regra usa operações aritméticas que exigem N conhecido. O Prolog não descobre automaticamente que N = 5, mesmo sabendo que
5
!
=

120

5!=120.

Saída do aritmetica.pl

```
--- = unifica, is calcula
?- A=2+3.
   2+3=2+3
?- A is 2+3.
   5 is 2+3
?- 5=:=2+3.
   5=:=2+3
?- 2+3=5.
   false.
?- A is 7 mod 3.
   1 is 7 mod 3
?- A is 7//2.
   3 is 7//2
?- A is 2^10.
   1024 is 2^10
--- fatorial, fib e soma
?- fatorial(5,A).
   fatorial(5,120)
?- fatorial(0,A).
   fatorial(0,1)
?- fib(10,A).
   fib(10,55)
?- soma([1,2,3,4],A).
   soma([1,2,3,4],10)
--- aritmetica nao e relacional
?- fatorial(A,120).
   ERRO: instantiation_error
?- A is B+1.
   ERRO: instantiation_error
?- A is 5*4+2.
   22 is 5*4+2
?- A=5*4+2.
   5*4+2=5*4+2
?- fatorial(10,A).
   fatorial(10,3628800)
?- fib(20,A).
   fib(20,6765)
?- fatorial(A,120).
   ERRO: instantiation_error
Ready
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= (unificação): verifica se dois termos podem ser unificados, sem fazer cálculos. Ex.: A = 2+3 → A recebe a expressão 2+3.

is (cálculo): calcula o lado direito e atribui o resultado ao lado esquerdo. Ex.: A is 2+3 → A = 5.

=:= (igualdade aritmética): calcula os dois lados e verifica se os valores são iguais. Ex.: 5 =:= 2+3 → verdadeiro.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

A regra do fatorial usa operações aritméticas, como N > 0 e N1 is N - 1, que exigem que N já tenha um valor conhecido.

Na consulta fatorial(N, 120), N começa desconhecido, causando um erro de instanciação.

Já concatena(X, [3], [1,2,3]) funciona porque a concatenação usa unificação e recursão, permitindo que o Prolog descubra X = [1,2] sem precisar calcular uma expressão aritmética.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

igual
Ambos usam recursão.

Ambos possuem um caso base: o fatorial de zero é 1.

Ambos calculam o fatorial multiplicando o número pelo fatorial do anterior.

Diferenças:

Prolog: usa fatos, regras e consultas. A aritmética exige operações explícitas, como is.

Haskell: usa definições de funções e equações, com avaliação de expressões diretamente.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X): funciona e retorna osvaldo, paulo e jake.

solteiro2(X): dá erro de instanciação, pois X está sem valor quando \+ casado(X) é executado.

maximo2(5, 3, M): dá duas respostas, M = 5 e M = 3, porque a segunda regra não verifica se Y é maior.

maximo3(5, 3, 3): dá false, porque a primeira regra é verdadeira (5 >= 3) e o corte ! impede tentar a segunda regra, que permitiria M = 3.

Saída de controle.pl e mensagem de ancestral_ruim

```
1
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(caio, ana)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(ana, caio)
   tenta caio e bia
```

```
    tenta caio e caio
2
ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores: existem 24 colorações válidas.

Com 2 cores: existem 0 colorações válidas.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
--- quantas coloracoes existem com 3 cores?
24
--- e com 2 cores?
?- mapa([vermelho,verde],A).
   false.
--- e com 4 cores?
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

As duas respostas foram:

mesmo_gosto(caio, ana)

mesmo_gosto(ana, caio)

O Prolog tentou as combinações de amigos na ordem em que os fatos foram definidos.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1(X) primeiro procura pessoas que satisfazem homem(X) e depois verifica se não são casadas. Retorna osvaldo, paulo e jake.

solteiro2(X) tenta verificar \+ casado(X) antes de atribuir um valor a X. Como a negação é avaliada sem uma pessoa específica, o resultado não serve para gerar todos os solteiros; em algumas implementações, a consulta pode até gerar erro de instanciação.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte (!) impede que o Prolog tente a segunda cláusula depois que a primeira condição é satisfeita. Mas, na primeira cláusula, o terceiro argumento é unificado com X antes de executar o corte.

Assim, em maximo3(5, 3, 3), a unificação exige que 3 = 5 e deveria falhar antes de chegar ao corte. Portanto, com o código mostrado, o resultado correto é false, não true.

maximo3(X, Y, M) :-

```
    X >= Y, !,
    M = X.
maximo3(_, Y, Y).
```

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_bom verifica primeiro se existe uma relação de progenitor e depois continua a busca recursivamente. Cada chamada avança pela árvore genealógica.

ancestral_ruim chama a si própria antes de verificar qualquer fato. Assim, entra em recursão infinita e acaba esgotando a pilha.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

3 cores: 24 colorações.

2 cores: 0 colorações.

4 cores: 384 colorações.
O programa é considerado declarativo porque descreve as condições que uma coloração válida deve satisfazer: regiões vizinhas precisam ter cores diferentes.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No Prolog, concatena(X, Y, [1,2,3]) encontra várias divisões possíveis da lista usando unificação e backtracking.
No Scheme, append recebe as listas e devolve a lista concatenada.
Assim, o lógico descreve relações e busca soluções, enquanto o funcional calcula valores por meio de funções.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

A ordem das metas influencia o resultado: em solteiro2(X), tentar a negação antes de atribuir um valor a X impede a busca correta pelos solteiros.

A recursão pode não terminar: ancestral_ruim chama a si própria sem avançar na busca e acaba estourando a pilha.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas que envolvem regras e relações, como sistemas especialistas, genealogias e resolução de restrições, porque ele permite gerar várias soluções automaticamente.

Não escolheria Prolog para aplicações que exigem interfaces gráficas complexas, controle detalhado do estado ou integração com sistemas corporativos, nas quais Java costuma oferecer ferramentas e bibliotecas mais adequadas.

Exercícios opcionais

Opcionais: resultado dos testes

```
Exercicio 1
   FALHOU  irma(ana, jake)
   FALHOU  irma(tereza, vern)
   FALHOU  NAO irma(jake, ana)
   FALHOU  tia(tereza, joana)
```

```
    FALHOU  tia(joana, leo)
    FALHOU  NAO tia(joana, jake)
Exercicio 2
    FALHOU  ultimo([a,b,c], c)
    FALHOU  ultimo([7], 7)
    FALHOU  NAO ultimo([a,b,c], b)
Exercicio 3
    FALHOU  soma_quadrados([1,2,3], 14)
    FALHOU  soma_quadrados([], 0)
    FALHOU  pares([1,2,3,4,5,6], [2,4,6])
    FALHOU  pares([1,3], [])
Exercicio 4
    FALHOU  mdc(12, 18, 6)
    FALHOU  mdc(17, 5, 1)
    FALHOU  mdc(10, 0, 10)
    FALHOU  potencia(2, 10, 1024)
    FALHOU  potencia(5, 0, 1)
Exercicio 5
    FALHOU  caminho(curitiba, foz)
    FALHOU  caminho(joinville, cascavel)
    FALHOU  NAO caminho(foz, curitiba)
    FALHOU  NAO caminho(curitiba, paris)
Exercicio 6
    FALHOU  3 cores: 12 coloracoes
    FALHOU  2 cores: 2 coloracoes
    FALHOU  1 cor: nenhuma
Exercicio 7
    FALHOU  colega(ana, bruno)
    FALHOU  NAO colega(ana, carla)
    FALHOU  NAO colega(ana, ana)
    FALHOU  ganha_mais(ana, bruno)
    FALHOU  NAO ganha_mais(elisa, ana)
    FALHOU  total_depto(ti, 12500)
    FALHOU  total_depto(rh, 9000)
Ready
```

E1 Regras de família: código escrito

```
irma(X, Y) :-
    mulher(X),
    mae(M, X), mae(M, Y),
    pai(P, X), pai(P, Y),
    X \== Y.

tia(X, Y) :-
    mulher(X),
    progenitor(P, Y),
    irmao(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :-
    ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :-
    soma_quadrados(T, S1),
    S is S1 + H * H.

pares([], []).
pares([H|T], [H|R]) :-
    0 is H mod 2,
    pares(T, R).
pares([H|T], R) :-
    1 is H mod 2,
    pares(T, R).
```

E4 Aritmética: código escrito

```
mdc(A, 0, A) :-
    A > 0.
mdc(A, B, D) :-
    B > 0,
    R is A mod B,
    mdc(B, R, D).

potencia(_, 0, 1).
potencia(B, E, R) :-
    E > 0,
    E1 is E - 1,
    potencia(B, E1, R1),
    R is B * R1.
```

E5 Grafo com ciclos: código escrito

```
estrada(curitiba, joinville).
estrada(joinville, cascavel).
estrada(cascavel, foz).

ligado(X, Y) :- estrada(X, Y).
ligado(X, Y) :- estrada(Y, X).

caminho(X, Y) :-
    caminho(X, Y, [X]).

caminho(X, Y, _) :-
    ligado(X, Y).

caminho(X, Y, Visitados) :-
    ligado(X, Z),
    \+ member(Z, Visitados),
    caminho(Z, Y, [Z|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Cores, [RS, SC, PR]) :-
    member(RS, Cores),
    member(SC, Cores),
    member(PR, Cores),
```

```
    RS \== SC,
    SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
funcionario(ana, ti, 7000).
funcionario(bruno, ti, 5500).
funcionario(carla, rh, 5000).
funcionario(elisa, rh, 4000).

colega(X, Y) :-
    funcionario(X, D, _),
    funcionario(Y, D, _),
    X \== Y.

ganha_mais(X, Y) :-
    funcionario(X, _, SX),
    funcionario(Y, _, SY),
    SX > SY.

total_depto(Depto, Total) :-
    findall(S, funcionario(_, Depto, S), Salarios),
    soma_lista(Salarios, Total).

soma_lista([], 0).
soma_lista([H|T], Total) :-
    soma_lista(T, Parcial),
    Total is H + Parcial.
```